



Programmazione asincrona

Gestire attivamente le attese

Programmi concorrenti

- Se la necessità di eseguire più operazioni, indipendenti tra loro, **in parallelo** porta alla creazione di più thread, altre esigenze di programmazione possono beneficiare di un approccio differente
 - l'uso dei thread trova piena giustificazione quando occorre eseguire algoritmi complessi, basati principalmente sull'uso intenso della CPU e si dispone di un hardware multicore
 - in queste situazioni, infatti, è possibile ridurre il tempo complessivo di elaborazione sfruttando il fatto che più computazioni procedono parallelamente
- Se il programma, invece, deve coordinare molteplici attività che si svolgono **al di fuori** di esso, il problema dominante diventa l'attesa dei risultati
 - Sebbene sia possibile gestire queste situazioni creando più thread, questo, in generale, non è conveniente (i costi legati alla programmazione multithread sono costituiti, da un lato, dalla complessità legata ai meccanismi di sincronizzazione e dall'altro, dalla necessità di allocare preventivamente lo stack di esecuzione di ciascun thread)
 - In caso di creazione di molti (1000+) thread, tale costo diventa significativo e, in certe situazioni, proibitivo

lezione del 26-05-2026

Ci sono certi casi in cui è utile e buona norma riorganizzare il codice sfruttando le tecniche della programmazione asincrona. Essa è un'alternativa alla programmazione concorrente, in cui avevamo un problema di una complessità tale che conveniva parallelizzarlo su sotto-elaborazione (idea di divide et impera). Ogni sotto-problema è assegnato ad un thread, e tutti i thread lavorano in parallelo. In questo discorso che avevamo fatto sulla programmazione concorrente assumevamo che necessariamente c'era il bisogno di una CPU con più core

L'introduzione dei thread avevamo visto comportare una complessità in più sull'architettura, ovvero la creazione di uno stack per ogni thread. C'è quindi un overhead a livello di sistema operativo, ma abbiamo anche il beneficio di avere le elaborazioni in parallelo.

questo approccio della programmazione concorrente in cui i tempi di esecuzione sono fortemente dipendenti dalla frequenza di clock della cpu. Altri tipi di applicazioni, chiamati I/O bound, si pongono spesso in attesa di qualcosa che può essere segnali dall'hardware (interfaccia con il file system nella lettura e scrittura di file) o attesa nella risposta che arriva da un altro processo

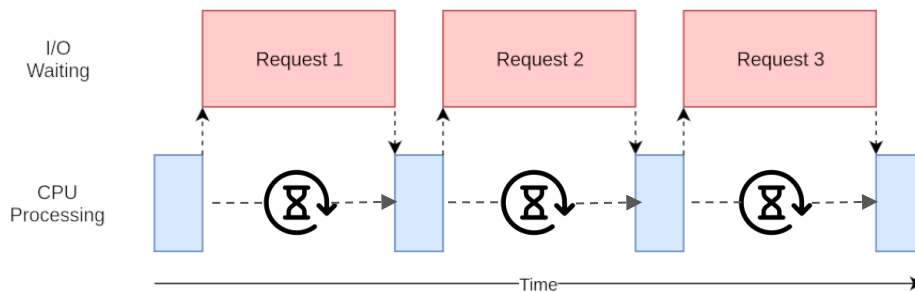
Esiste la possibilità che un processo si ponga in una condizione di attesa di una risposta che arriva da un altro processo e quindi esegue una procedura bloccante.

Operazioni bloccanti

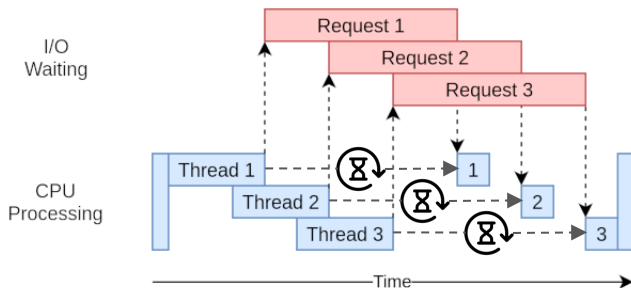
a volte i thread rimangono in attesa di un qualcosa che non dipende strettamente da lui; ad esempio abbiamo l'input-output o l'inter process communication

- I casi in cui l'approccio multi-thread può non essere ottimale sono quelli in cui la computazione richiama di ricevere informazioni da un sottosistema separato (che non è sotto il controllo del processo in esecuzione)
 - Come il file system, la rete, un timer, un altro programma...
- In queste situazioni l'esecuzione non può continuare e occorre attendere che il sottosistema in questione fornisca le informazioni attese
 - Normalmente, il sistema operativo rileva la situazione (implementa system call bloccanti) e sposta il thread corrente nello stato *"NotRunnable"*, sospendendone l'esecuzione fino a che non si verifica la condizione attesa
- **Se il programma deve svolgere altri compiti**, oltre a quello che ha generato l'attesa, si creano tre possibilità:
 - Attendere: gli altri compiti saranno eseguiti successivamente, dal thread corrente
 - Multithread: si creano più thread secondari per eseguirli, accollandosi la complessità legata alla loro sincronizzazione
 - Programmazione Asincrona: si organizza il codice in modo tale da separare la richiesta di eseguire l'operazione, dalle operazioni che dovranno essere fatte quando arriverà la risposta, così da non bloccare l'esecuzione del thread corrente, ammesso che sia altro da fare

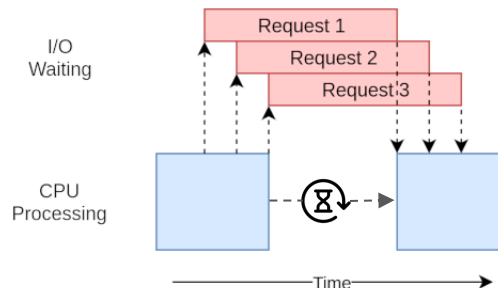
Strategie di esecuzione



(a) Single-threaded process



(b) Multi-threaded process
with GIL acquired by current thread



(c) Single-threaded process
with asyncio

Costo di creare n
thread e necessità
di sincronizzarsi

Tre strategie per gestire l'attesa

Se il programma ha altro da fare durante l'attesa, ci sono tre strategie possibili:

Strategia	Idea	Vantaggi	Limiti
Sequenziale (bloccante)	Aspetto e poi continuo	Semplice Facile da ragionare	Spreca tempo di CPU Nessuna sovrapposizione
Thread multipli (preemptive)	Delego l'operazione bloccante ad altri thread	Parallelismo reale Sfrutta multicore	Costo elevato (stack, scheduling) Sincronizzazione complessa
Asincronia (cooperativo)	Separo richiesta e continuazione	Non blocca il thread Scala a migliaia di operazioni	Più complesso concettualmente Richiede supporto di esecuzione

Operazioni bloccanti e attesa

- La **domanda di progetto**: il collo di bottiglia è la CPU o l'attesa (I/O, rete, timer)?
 - CPU-bound → parallelismo con thread
 - I/O-bound → concorrenza con esecuzione asincrona

Programmazione Concorrente: mentre faccio questo faccio anche quello

Programmazione Asincrona: ne faccio un po', poi faccio altro e torno in un tempo successivo a continuare

Esecuzione asincrona

Si usano i **thread** quando occorre elaborare in parallelo, mentre si usa l'**esecuzione asincrona** quando occorre attendere in parallelo

L. Palmieri, Zero to production in Rust, 2022, ISBN 979-8847211437



Il costo dei thread

- Ogni thread del sistema operativo ha un costo fisso, indipendente dal lavoro che svolge
 - **Stack pre-allocato**: tipicamente da 1 a 8 MB di spazio di indirizzamento riservato per thread
 - **Strutture del kernel**: descrittore del thread, slot nello schedulatore, contabilità
 - **Cambio di contesto**: salvataggio/ripristino dei registri, invalidazione di cache e TLB ($\sim 1-2 \mu s$)
- Finché i thread sono poche decine, questi costi sono trascurabili
 - Diventano critici quando se ne vogliono creare molti (1000+), come in un server di rete
 - Esempio: 10.000 connessioni simultanee con un thread ciascuna $\rightarrow$ diversi GB solo di stack
- Il paradosso: gran parte di questi thread sarebbe **bloccata in attesa**, senza usare CPU
 - Si paga il costo di un thread completo per fargli fare, di fatto, nulla

Prestazioni a confronto

- I costi di creazione e cambiamento di contesto tra attività asincrone e thread sono analizzati nel sito
 - <https://github.com/jimblandy/context-switch>
- Il sito riporta la metodologia, il codice e i risultati ottenuti su un elaboratore specifico, con il relativo sistema operativo
 - Gli ordini di grandezza sono comunque interessanti

Operazione	async	thread
Creazione di task	0.3 μ s	17 μ s
Cambio di contesto	0.2 μ s	1.7 μ s
Uso di memoria	300÷500 Byte	> 9.5 KByte

Esecuzione asincrona

- Il modo più diretto di implementare la terza strategia richiede che le azioni conseguenti ad una data operazione bloccante siano racchiuse in un'apposita funzione (**callback**)
 - Tale funzione riceve come parametro il risultato dell'operazione bloccante e sarà invocata quando il sottosistema che è stato interrogato avrà fornito la propria risposta
- Chi si occupa di fornire tale valore e in quale thread avverrà la chiamata?
 - Linguaggi diversi offrono soluzioni alternative
- In Javascript, ad esempio, il modello di esecuzione prevede la presenza di una coda dei messaggi (come parte integrante del linguaggio oltre a stack e heap), in cui il driver del sottosistema interrogato provvede ad inserire il risultato
 - Il driver è eseguito in un thread separato
 - Il thread principale esegue costantemente un ciclo, in cui attende la presenza di un messaggio, lo estrae dalla coda e lo elabora
 - Quando la risposta arriverà, questa verrà naturalmente elaborata dal ciclo di elaborazione dei messaggi

Separare richiesta e continuazione

- Nell'approccio asincrono
 - Avvio un'operazione
 - NON attendo il risultato
 - Specifico cosa fare quando il risultato arriva (continuazione)
- Il modo più semplice per indicare la continuazione è una callback
 - Pseudo-codice: `read_async(file, |result| { process(result); });`
- Timeline
 - t_0 : invoco `read_async(...)`
 - t_1 : il programma continua
 - t_2 : arriva il risultato
 - t_3 : viene eseguita la callback
- Vantaggio: il thread NON resta bloccato dall'attesa e può svolgere altro lavoro

Continuazioni e callback

- Un tipico modo per implementare operazioni asincrone è basarsi su API ad eventi, ovvero funzioni che permettono di richiedere ad uno **strato sottostante** (come il sistema operativo o una piattaforma di esecuzione) l'operazione che si intende eseguire, passando una **callback** che dovrà essere invocata nel momento in cui lo strato sottostante avrà terminato la sua parte di esecuzione

Cessa di essere bloccante, ma ritorna immediatamente

```
val f1: AsyncRead = ...  
val f2: AsyncRead = ...  
read_async(f1, vec![], |buffer: &[u8]|{  
    // process buffer from file1...  
});  
read_async(f2, vec![], |buffer: &[u8]|{  
    // process buffer from file2  
});
```

Funzione Callback riceverà come parametro l'esito dell'operazione bloccante

L'inferno delle callback

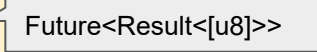
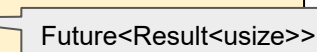
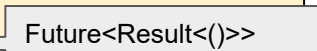
- Questo modo di scrivere il codice, tuttavia, crea grossi problemi quando l'azione che si sta compiendo richiede, **in cascata**, una seconda azione asincrona
 - Occorre infatti che la callback indicata provveda ad invocare un'ulteriore operazione passando la relativa callback
- Le cose si complicano se tale operazione è ciclica e se occorre gestire eventuali errori
 - Occorre trasformare il codice in una macchina a stati finiti
- Gli errori possono originarsi in momenti molto diversi
 - All'atto dell'invocazione della funzione asincrona
 - Come conseguenza dell'elaborazione asincrona

```
let h1 = open_file_async("f1", FileMode::read )?;
let h2 = open_file_async("f2", FileMode::write)?;
let mut buffer = vec![];
read_async(h1, &mut buffer, |res1|{
    if (res1.is_ok()) {
        write_async(h2, res1.unwrap(), |res2| {
            if (res2.is_ok()) {
                //scrittura completata con successo
            } else {
                //scrittura fallita
            }
        });
    } else {
        //lettura fallita
    }
})?; // impossibilità di leggere
//Quando il programma arriva qui, non è ancora
//stato fatto nulla!
```

Esecuzione parziale

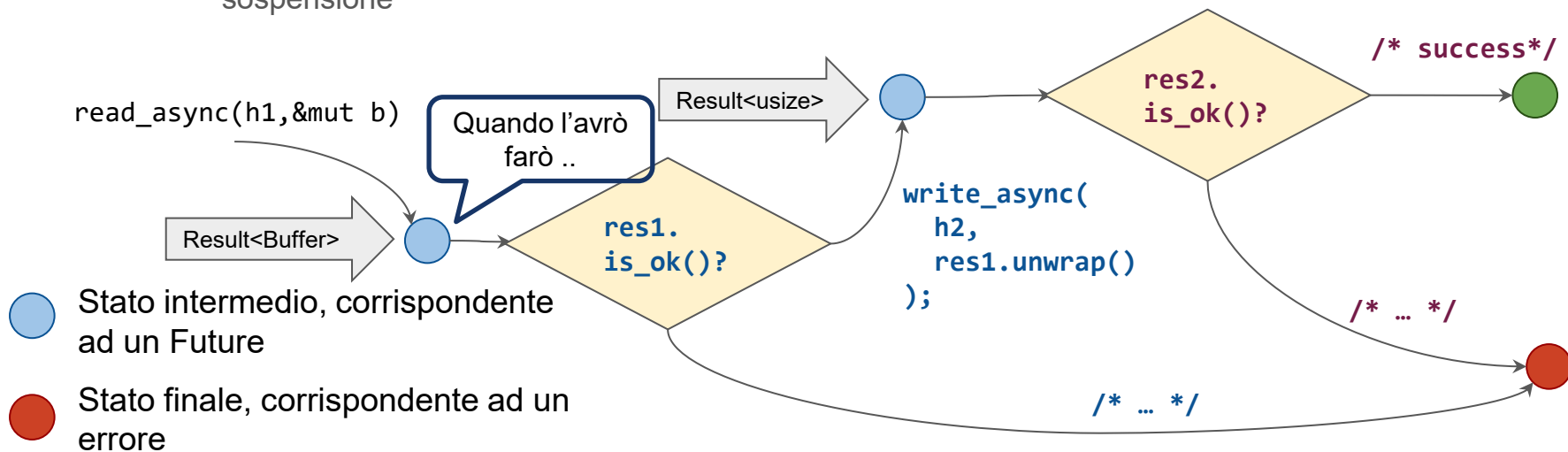
- Un primo passo che aiuta a semplificare il problema è provare a riscrivere la forma **annidata** delle callback in una forma **lineare** (riscriviamo le call back **una a fianco all'altra**)
 - Appoggiandosi ad una struttura dati che mantenga le informazioni di stato (**future**) che **incapsula il risultato che verrà**

```
read_async(h1, &mut buffer, |res1|{  
  if (buffer.is_ok()) {  
    write_async(h2, res1.unwrap(), |res2| {  
      if (res2.is_ok()) { /* success */ }  
      else { /* ... */ }  
    });  
  } else {  
    //lettura fallita  
  }  
})?;
```

```
read_async(h1, &mut buffer)   
  .and_then( |res1|{  
    if (res1.is_ok()) {  
      write_async(h2, res1.unwrap());  
    } else { /* ... */ }   
  }  
  .and_then(|res2| {  
    if (res2.is_ok()) { /* success */ }  
    else { /* ... */ }   
  })  
  .map_error( |err| { /* ... */ });
```

Esecuzione parziale

- Questa forma aiuta a vedere come le operazioni che si intendono eseguire possano essere viste come **una macchina a stati finiti**, che evolve "a strappi"
 - Quando raggiunge uno stato intermedio ritorna e, per continuare, sarà necessario riprenderne l'esecuzione passando come parametro l'esito dell'operazione asincrona che ha causato la sospensione

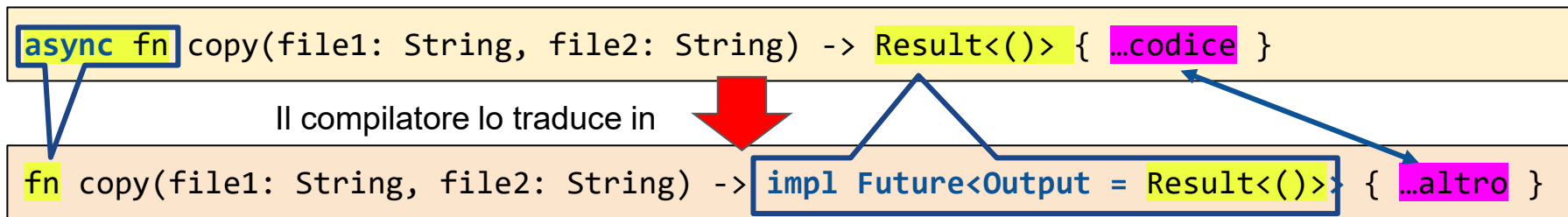


Esecuzione parziale

- La macchina a stati finiti può essere implementata da una chiusura
 - Essa racchiude il proprio **stato** (il pallino della slide precedente) e tutte le **variabili locali** di cui l'esecuzione ha bisogno
 - Quando viene eseguita ritorna un valore che indica se ha raggiunto uno degli stati finali o se si trova ancora in uno stato intermedio
 - Questo permette di non bloccare il thread corrente e procedere con l'esecuzione di altri task
- I punti di blocco sono tutti in corrispondenza degli stati intermedi
 - Questi sono immediatamente preceduti dall'invocazione di una funzione asincrona
 - Quando uno stato intermedio viene raggiunto, la chiusura ritorna
- Perché la macchina a stati possa progredire occorrono due condizioni:
 - Che la funzione asincrona invocata scateni qualche attività (in che thread avviene?) che possa portare all'evoluzione dello stato (ossia bisogna eseguire `write_async`, nell'esempio)
 - Che ci sia qualcuno che indichi che l'attesa è terminata e ci sono le condizioni affinché lo stato possa evolvere (ossia bisogna sapere che il Future si è materializzato).

Async e await

- Il compilatore Rust supporta esplicitamente la programmazione asincrona grazie all'introduzione di due parole chiave nel linguaggio (**async** e **await**) ed alla presenza di un tipo specifico (**Future**) nella libreria core
 - Se una funzione o un blocco di codice sono preceduti dalla parola chiave **async**, il compilatore ne analizza il contenuto e lo trasforma in una macchina a stati
 - La funzione o il blocco vengono ridotti all'inizializzazione di tale macchina a stati
 - Il tipo restituito (`impl Trait`) viene trasformato da **T** in un **tipo anonimo** (ossia non ne conosciamo il tipo, ma sappiamo solo che implementa il tratto `Future`) che implementa il tratto **Future** al cui interno è implementata la macchina a stati precedentemente sintetizzata



Async e await

- Se all'interno del codice della funzione è presente una chiamata ad un'altra funzione asincrona, per poter accedere al suo risultato occorre esplicitamente attenderlo, attraverso l'operatore `.await`
 - Questo introduce un nuovo stato all'interno della macchina a stati associata alla funzione

```
async fn copy(h1: FileHandle, h2: FileHandle) -> std::io::Result<()> {  
    let mut buffer = vec![];  
    h1.read_async(&mut buffer).await?; // quando si incontra il .await il thread passa  
                                        // a fare altro (se c'è altro da fare)  
                                        // quando il Future si materializza si va a  
                                        // vedere il tipo di risultato  
                                        // Se Error => si esce, altrimenti si continua  
  
    h2.write_async(&buffer).await  
}
```

Il tratto Future

```
use std::pin::Pin;
use std::task::{Context, Poll};

pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context)
        -> Poll<Self::Output>;
}
```

Il metodo poll
restituisce lo stato: è
finito, oppure stato
intermedio

```
pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

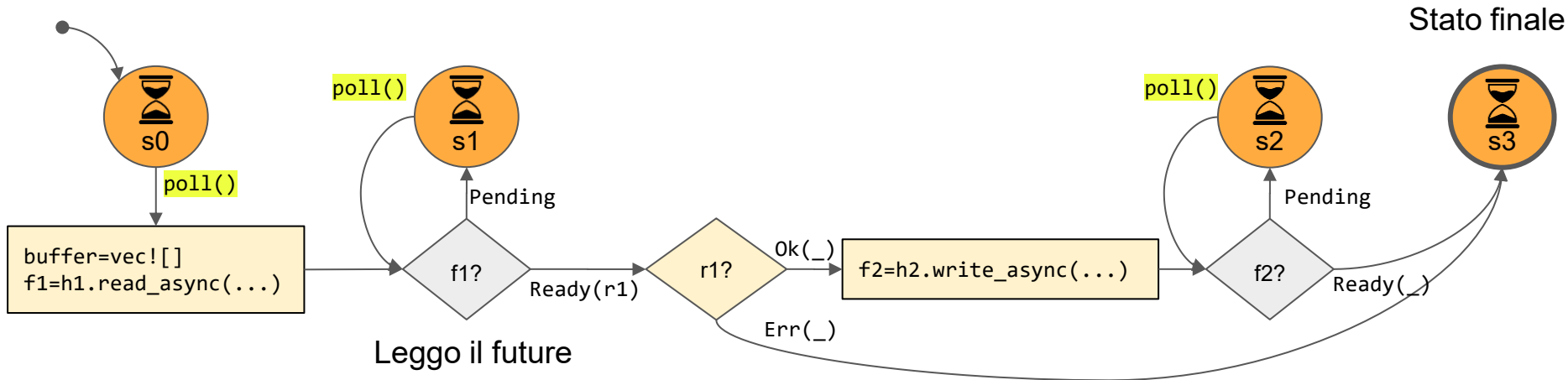
- **Pin<T>** è un particolare smart pointer che impedisce che la struttura venga mossa
 - Permettendo l'uso di riferimenti relativi e autoreferenziali
- **Context** incapsula un oggetto di tipo **Waker**, mediante il quale è possibile notificare all'esecutore quando il metodo **poll(...)** potrà essere richiamato

Il tratto Future

- Tratto che viene implementato dagli oggetti che descrivono una computazione asincrona
 - Tale computazione può essere ancora in corso o essere terminata
- Il tratto offre un singolo metodo, **poll(...)**, che implementa la logica della macchina a stati associata alla funzione
 - L'oggetto che implementa il tratto mantiene al suo interno lo stato corrente
- Il metodo **poll(...)** restituisce enum che può assumere due soli valori:
 - **Poll:Pending** - per indicare che la computazione è ancora in corso
 - **Poll::Ready(val)** - per indicare che la computazione è terminata e ha come risultato **val**
- Un oggetto che implementa questo tratto è inerte (**lazy**)
 - Affinché la computazione proceda, occorre che qualcuno ne invochi il metodo **poll(...)**

Generare la macchina a stati

```
async fn copy(h1: FileHandle, h2: FileHandle) -> std::io::Result<()> {  
    let mut buffer = vec![];  
    h1.read_async(&mut buffer).await?;  
    h2.write_async(&buffer).await  
}
```



Generare la macchina a stati

- Per ciascuno stato individuato, il compilatore sintetizza una struct in cui memorizzare le variabili locali necessarie per consentire l'esecuzione

```
struct S0 {  
    h1: FileHandle,  
    h2: FileHandle,  
}
```

```
struct S1 {  
    h2: FileHandle,  
    buffer: Vec<u8>,  
    f1: impl Future<Output=Result<usize>>,  
}
```

```
struct S2 {  
    f2: impl Future<Output=Result<usize>>,  
}
```

```
struct S3 {}
```

Generare la macchina a stati

- Inoltre genera una enum che racchiude i possibili stati e implementa il tratto Future

```
enum CopySM {  
    s0(S0),  
    s1(S1),  
    s2(S2),  
    s3(S3)  
}
```

```
impl Future for CopySM {  
    type Output = std::io::Result<>  
    fn poll(self: Pin<&mut Self>, cx: &mut Context)  
        -> Poll<Self::Output> {  
        loop {match self {  
            CopySM::s0(state) => { ... },  
            CopySM::s1(state) => { ... },  
            CopySM::s2(state) => { ... },  
            CopySM::s3(state) => { ... },  
        } }  
    }  
}
```


Generare la macchina a stati

```
CopySM::s0(state) => {  
  let mut buffer = vec![];  
  let f1 = state.h1.read_async(&mut buffer);  
  let state = S1 {h2: state.h2, buffer, f1 };  
  *self = CopySM::S1(state);  
}
```

Generare la macchina a stati

```
CopySM::s1(state) => {  
  match (state.f1.poll(cx) {  
    Poll::Pending => return Poll::Pending,  
    Poll::Ready(r1) =>  
      if r1.is_ok() {  
        let f2 = state.h2.write_async(&mut state.buffer);  
        let state = S2{ f2 };  
        *self = CopySM::s2(state);  
      } else {  
        *self = CopySM::s3(S3);  
        return Poll::Ready(r1);  
      }  
  }  
}
```

Generare la macchina a stati

```
CopySM::s2(state) => {  
  match (state.f2.poll(cx) {  
    Poll::Pending => return Poll::Pending,  
    Poll::Ready(r2) =>  
      *self = CopySM::s3(S3);  
      return Poll::Ready(r2);  
  }  
}
```

```
CopySM::s3(_) => {  
  panic!("poll() was invoked again after Poll::Ready has been returned");  
}
```

Implementare la funzione

- Il codice generato dal compilatore per la funzione si riduce all'inizializzazione della macchina a stati

```
fn copy(h1: FileHandle, h2: FileHandle) ->
    impl Future<Output = std::io::Result<>> > {
    CopySM::s0(
        s0 { h1, h2 }
    )
}
```

La parola chiave async

- Anteporre **async** a una funzione (o a un blocco) ne cambia radicalmente la semantica
 - Il compilatore analizza il corpo e lo trasforma in una macchina a stati finiti
 - Il tipo di ritorno passa da **T** a un tipo anonimo che implementa il tratto **Future<Output=T>**
 - Il corpo della funzione viene ridotto alla sola inizializzazione di quella macchina a stati
- Conseguenza cruciale: **chiamare una funzione async NON ne esegue il corpo**
 - Restituisce solo un **Future** inerte, nel suo stato iniziale
 - È un valore che descrive una computazione da fare, non la computazione già avviata

L'operatore `.await`

- Chi invoca una funzione asincrona, ottiene come risultato un `Future` nel proprio stato iniziale e affinché possa capitare qualcosa, occorre che ne venga invocato il metodo `poll(...)`
- Per ottenere il risultato di un `Future`, occorre attenderlo esplicitamente con `.await`
 - con il metodo `.await()` ci garantisce che viene chiamato `poll()`
 - Senza `.await` il `Future` resta inerte e non viene mai eseguito (il compilatore lo segnala con un warning)
- Semantica di `.await`
 - Se il valore non è ancora disponibile, la funzione corrente si **sospende** e restituisce il controllo
 - Il thread non resta bloccato: è libero di eseguire altri task pronti
 - Quando il valore arriva, l'esecuzione riprende esattamente dal punto di sospensione
- Ogni `.await` introduce un nuovo stato nella macchina a stati della funzione
 - segnala dove sospendere e riprendere
 - L'operatore `?` si combina con `.await` per propagare gli errori in modo lineare.

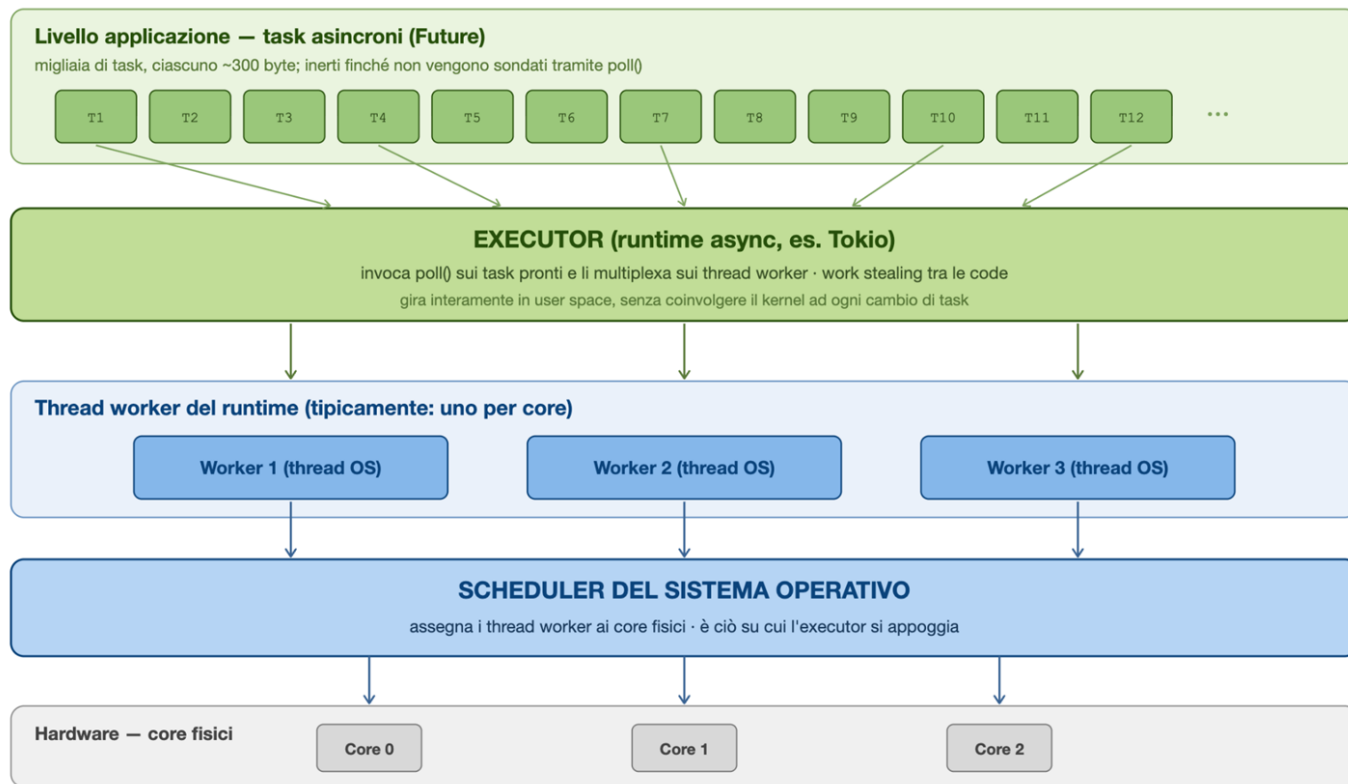
Possesso nel tempo

- Nel codice sincrono, il possesso è legato prevalentemente allo stack corrente
- Nel codice asincrono, una computazione può sospendersi e riprendere più tardi
 - Le variabili usate dopo `.await` diventano parte dello stato del **Future**
 - Se l'esecutore è multithread, quel **Future** può essere spostato tra thread differenti
- Nei programmi asincroni, il possesso attraversa il tempo, non solo lo spazio
 - I dati usati dopo `.await` devono restare validi
 - I task spostabili tra thread devono avere il tratto **Send**
 - I riferimenti condivisi tra thread devono avere il tratto **Sync**
 - La creazione di task indipendenti spesso richiede la proprietà dei dati

Un future è inerte: serve un esecutore

- Il compilatore genera i tipi della macchina a stati e riscrive le funzioni, ma il linguaggio si ferma qui
 - Nessun supporto precostituito per **eseguire** effettivamente i **Future**
 - lascia il compito al programmatore di richiamare `poll` quando è ora per fare in modo che il programmatore adotti una strategia che si addatta al tipo di contesto
 - Il programmatore può scegliere quale libreria adottare nel proprio progetto, in base alle specifiche necessità
- Affinché una computazione asincrona proceda, qualcuno deve farne avanzare la macchina a stati
 - Se viene chiamata una funzione `async` dentro un'altra funzione `async`, il suo **Future** viene automaticamente a far parte della macchina a stati del chiamante
 - Se viene chiamata da una funzione normale, occorre gestire il **Future** esplicitamente con un **executor**
- L'executor è uno **schedulatore cooperativo** a livello applicazione
 - Mantiene i task pronti e ne fa avanzare la macchina a stati invocando il metodo `poll()` del **Future**
 - A differenza dello scheduler del S.O. non può sottrarre il controllo ad un task, che deve cederlo spontaneamente ad un punto di attesa (`.await`)

Esecutori



1

2

3

Chi invoca poll(): executor, reactor, Waker

- L'**executor** mantiene la coda dei task pronti e ne invoca il metodo **poll()** in un ciclo
 - Se **poll()** restituisce **Pending**, l'executor mette il task da parte e passa al successivo
 - Comportamento cooperativo, non preemptive: un task che non raggiunge mai **.await** monopolizza il worker
 - Il problema: come sa l'executor quando riprovare, senza fare polling attivo a vuoto?
- Risposta: il **Waker** contenuti nel **Context**
 - Prima di restituire **Pending**, il **Future** registra il proprio **Waker** presso il sottosistema (es, il reactor I/O)
 - Il reactor osserva gli eventi del sistema operativo (epoll, kqueue, IOCP via mio)
 - Quando l'evento atteso si verifica, il reactor risveglia il task, chiamando **wake()** sul **Waker**
 - Il task torna nella coda dei pronti e l'executor lo ri-campiona: la macchina a stati avanza
- È un modello ad eventi: non vengono sprecati cicli di CPU durante le attese

Esecutori

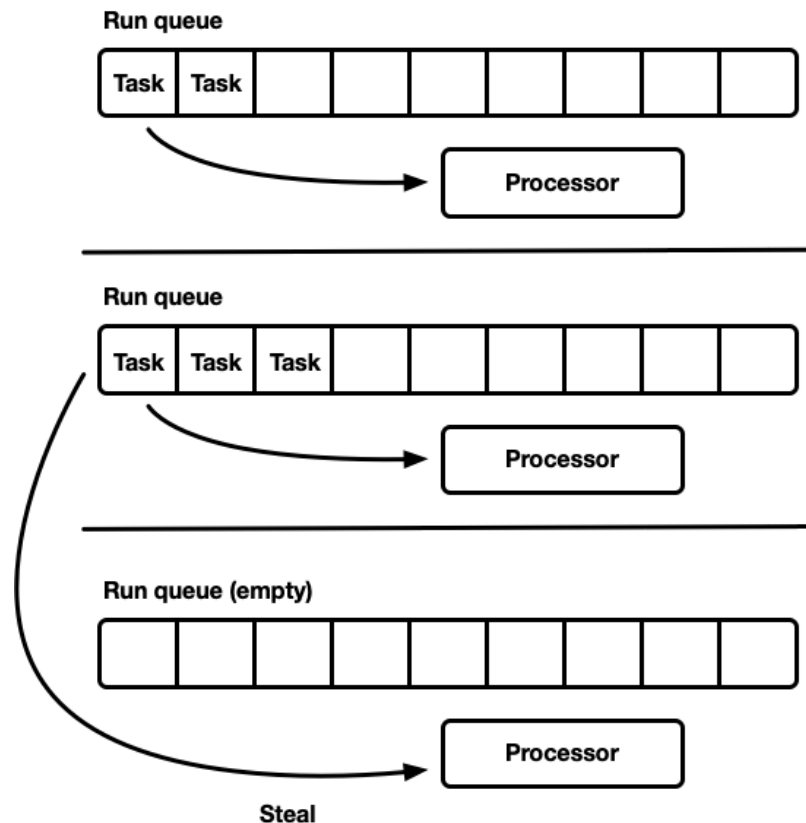
- Per far funzionare un programma Rust asincrono occorre aggiungere una libreria di supporto all'esecuzione, a scelta del programmatore
 - **tokio** - la più diffusa: rete, file, timer, sincronizzazione, canali; basata sul crate **mio** (Metal I/O)
 - **smol** - leggera, adatta a sistemi embedded
 - **async-std** - controparte asincrona della libreria standard
- I diversi ambienti di esecuzione non sono equivalenti
 - **Tokio** implementa un ciclo reattivo proprio, basato sul *crate* ad alte prestazioni **mio** (Metal I/O), che non è compatibile con i tratti usati dagli altri due



- La programmazione asincrona in Rust consente di scrivere codice che può gestire molte operazioni simultanee in modo efficiente senza bloccarsi su operazioni I/O lente (come le richieste di rete o l'accesso ai file). Ciò significa che il programma può eseguire più operazioni in parallelo senza dover attendere che ogni operazione si completi prima di passare alla successiva.
- La programmazione asincrona in Rust è basata sui seguenti concetti:
 - **Future**: Una Future rappresenta un valore che potrebbe non essere ancora disponibile. Una Future in Rust è un oggetto che implementa il trait Future, il quale ha un unico metodo poll che viene chiamato per tentare di avanzare l'esecuzione della Future
 - **Async/Await**: Le keyword `async` e `await` semplificano l'uso delle Future. Una funzione definita come `async fn` restituisce automaticamente una Future. La keyword `await` viene usata per attendere il completamento di una Future all'interno di una funzione `async`. Quando una funzione asincrona è sospesa, il runtime può passare a eseguire altre operazioni asincrone.
 - **Executor**: Un Executor è responsabile di orchestrare e coordinare i vari Future e le funzioni asincrone. Il crate **tokio** è uno degli executor più popolari in Rust per la gestione della programmazione asincrona.

Architettura di elaborazione

- Tokio è multi-thread: costruisce tanti thread (thread-pool) quanti sono i core effettivi della CPU (default) e ripartisce l'esecuzione dei vari blocchi dei Future sui vari thread:
 - ogni coda è gestita da un ciclo reattivo (Processor) che estrae ed esegue i task presenti
 - quando un Processor esaurisce i task della propria coda, prova a rubarne alcuni ad altre code (work stealing), su base euristica
- L'intero algoritmo è ottimizzato per ridurre al minimo la sincronizzazione
- <https://tokio.rs/blog/2019-10-scheduler>



Vincoli del Thread-Pool

- Può succedere che l'elaborazione di una funzione asincrona inizi in un thread ma sia continuata in un thread differente
- Questo implica che tutti i valori utilizzati nella funzione asincrona il cui uso si estende in più stati devono implementare il tratto **Send**, mentre i riferimenti devono implementare il tratto **Sync**

Impostare un progetto con Tokio

- Si aggiunge **tokio** come dipendenza nel file **Cargo.toml**, scegliendo quali funzionalità attivare
 - Si annota il punto di ingresso del programma con **#[tokio::main]**, che avvia la libreria di esecuzione e rende la funzione **main()** **async**
- La proprietà **flavor** determina la struttura dell'esecutore
 - **multi_thread** (default): un pool di thread worker, tipicamente uno per core
 - **current_thread**: un solo thread esegue tutti i task; utile per test e ambienti vincolati

```
# Cargo.toml
[dependencies]
tokio = { version = "1", features = ["full"] }
```

```
#[tokio::main(flavor = "multi_thread", worker_threads = 4 )]
async fn main() { /* creazione di future e attesa relativa */
}
```

- Una funzione **async** può essere chiamata solo dentro una funzione **async**.

Attendere un Future: esecuzione sequenziale

- Il modo più semplice di usare un **Future** è attenderlo con **.await** dentro un blocco **async**
 - Due **.await** consecutivi creano una sequenza: il secondo parte solo quando il primo è completo
 - Durante ogni attesa **il thread è libero** di servire altri task, ma **questa funzione non avanza**
 - È il comportamento corretto quando il secondo passo **dipende** dal risultato del primo

```
async fn fetch_user(id:u64) -> User { /* ... */ }
async fn fetch_orders(user: &User) -> Vec<Order> { /* ... */ }

#[tokio::main]
async fn main() {
    let user = fetch_user(42).await;          // prima questo...
    let orders = fetch_orders(&user).await; // ...poi questo (dipende da user)
    println!("{}", user.name, orders.len());
}
```


tokio::spawn - avviare task concorrenti

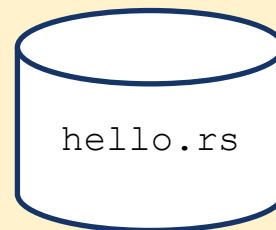
- **tokio::spawn(future)** consegna un **Future** all'esecutore come **task indipendente**
 - A differenza di **.await**, l'esecuzione inizia subito e in modo concorrente, senza sospendere il chiamante
 - Restituisce un **JoinHandle<T>**, che rappresenta il task e permette di recuperarne il risultato o di cancellarlo
- **tokio::task::spawn(f: T) -> JoinHandle<T::Output>**
 where T: Future + Send + 'static,
 T::Output: Send + 'static
- Vincoli sul tipo: il **Future** deve essere **Send + 'static**
 - **Send** perché il task può migrare tra i thread worker (work stealing)
 - **'static** non vuol dire che i dati vivono per sempre, ma che non ha riferimenti con tempo di vita (non statici) e dunque possiede tutti i dati che sta utilizzando
 - Si usa **move** per trasferire la proprietà dei dati al task
- **.await** = *"aspetto questo"* ⇔ **spawn(...)** = *"fa' questo mentre faccio altro"*

Task in Tokio

```
#[tokio::main]
async fn main() {
    let task = tokio::spawn(async { println!("Hello from Tokio!"); });

    thread::sleep(Duration::from_nanos(5));
    println!("Sayonara !");

    task.await.unwrap(); // aspetto l'esito del task
}
```



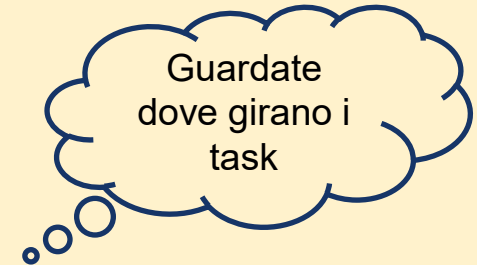
Gestione del tempo (I)

- La funzione `tokio::time::sleep(d: Duration).await` sospende l'esecuzione del task corrente per un tempo pari alla durata indicata
 - Durante l'attesa non viene consumata nessuna risorsa, se non la memoria necessaria a descrivere l'oggetto Future corrispondente
 - Allo scadere del tempo, l'esecuzione procede normalmente.

```

use std::thread;
use tokio::time::{sleep, Duration};
async fn countdown(start: u64, delay_ms: u64) {
    for _ in (0..=start).rev() { // Conto alla rovescia da start a 0
        sleep(Duration::from_millis(delay_ms)).await; // Attende in modo non bloccante
    }
}
#[tokio::main]
async fn main() {
    let task1 = tokio::spawn(async {
        println!("T1 sta girando sul thread: {:?}", thread::current().id());
        countdown(3, 200).await;
        println!("T1 sta girando sul thread: {:?}", thread::current().id());
    });
    let task2 = tokio::spawn(async {
        println!("T2 sta girando sul thread: {:?}", thread::current().id());
        countdown(5, 500).await;
        println!("T2 sta girando sul thread: {:?}", thread::current().id());
    });
    let task3 = tokio::spawn(async {
        println!("T3 sta girando sul thread: {:?}", thread::current().id());
        countdown(2, 1000).await;
        println!("T2 sta girando sul thread: {:?}", thread::current().id());
    });
    task1.await.expect("Task 1 failed");
    task2.await.expect("Task 2 failed");
    task3.await.expect("Task 3 failed");
}

```



```

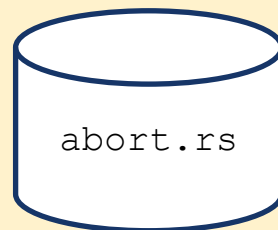
T2 sta girando sul thread: ThreadId(10)
T1 sta girando sul thread: ThreadId(5)
T3 sta girando sul thread: ThreadId(15)
T1 sta girando sul thread: ThreadId(3)
T3 sta girando sul thread: ThreadId(3)
T2 sta girando sul thread: ThreadId(3)

```

Cancellare un task

- La struct **JoinHandle** restituita da **spawn(...)** offre il metodo **abort()** per cancellare il task
 - **abort()** non è come killare un thread di sistema
 - La cancellazione ha effetto quando il task è sospeso su **.await**
 - Se il task è dentro un ciclo attivo, l'interruzione avviene al prossimo **.await**
 - Invocare **abort()** su un task già terminato non ha effetto: chiamate multiple equivalgono ad una sola
- Di norma, dopo **abort()** si attende la **JoinHandle** per verificare l'effettivo arresto
 - Si ottiene un **JoinError** con **is_cancelled() == true**
 - Se l'esecutore viene terminato, tutti i task in corso vengono cancellati

```
use tokio::time::{sleep, Duration};
#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        loop {
            println!("Task in esecuzione...");
            sleep(Duration::from_secs(1)).await;
        }
    });
    sleep(Duration::from_secs(3)).await;
    // Abortisce il task
    handle.abort();
    match handle.await {
        Ok(_) => println!("Task terminato normalmente"),
        Err(e) if e.is_cancelled() => println!("Task abortito"),
        Err(e) => println!("Task fallito: {e}"),
    }
}
```



Quando viene abortito il task ?

```
use tokio::time::{sleep, Duration};
```

```
#[tokio::main]
```

```
async fn main() {
```

```
    let handle = tokio::spawn(async {  
        println!("Task partito");  
        // Qui il task si sospende su .await  
        sleep(Duration::from_secs(10)).await;
```

```
        // Questa riga NON verrà eseguita se arriva abort() prima  
        println!("Task completato");  
    });
```

```
    sleep(Duration::from_secs(1)).await;
```

```
    handle.abort();
```

Quando viene abortito il task ?

```
    match handle.await {  
        Ok(_) => println!("Task terminato normalmente"),  
        Err(e) if e.is_cancelled() => {  
            println!("Task cancellato correttamente");  
        }  
        Err(e) => {  
            println!("Task fallito: {:?}", e);  
        }  
    }  
}
```



Non perdere il controllo dei task (II)

- Quando si hanno molti **task omogenei** è possibile usare strutture dedicate come

JoinSet<T>

- Offre il metodo `spawn(...)` che automaticamente memorizza la `JoinHandle` corrispondente
- Permette di iterare su tutte le handle per attenderne la terminazione: `join_next()` aspetta che **uno qualsiasi** dei task nel set finisca e restituisce il suo risultato. Quando non ci sono più task, restituisce `None` (rispetto a tenere tanti `JoinHandle` in un `Vec`, `JoinSet` è più comodo quando ci sono molti task e si vogliono raccogliere i risultati appena arrivano).

```
use tokio::task::JoinSet;
use tokio::time::{sleep, Duration};
#[tokio::main]
async fn main() {
    let mut set = JoinSet::new();
    for i in 0..5 {
        set.spawn(async move {
            sleep(Duration::from_millis(100 * i)).await;
            format!("task {i} finito")
        });
    }
    while let Some(result) = set.join_next().await {
        match result {
            Ok(value) => println!(" {value} "),
            Err(err) => println!("Task fallito: {err}"),
        }
    }
}
```



Attendere la terminazione di più Future

- Macro per coordinare il comportamento dei task
- **tokio::join!(f1, f2, ...)** attende il completamento di **tutti** i **Future** passati
 - li fa progredire in modo interlacciato **sullo stesso task**: non viene eseguito **spawn()**, non viene richiesto il tratto **Send**
 - Restituisce una tupla contenente i risultati, nell'ordine dei parametri passati
 - È il meccanismo corretto quando i compiti sono **indipendenti** e servono tutti i risultati
- **try_join!(...)** è la variante per **Future** che restituiscono **Result<T, E>**
 - Ritorna **Ok((r1,r2, ...))** se tutti hanno successo, oppure si interrompe al **primo errore** restituendolo

Attendere la terminazione di più Future

```
use tokio::time::{sleep, Duration};

// Una funzione asincrona che attende per un certo periodo
async fn task(name: &str, duration: u64) {
    println!("Inizio task: {}", name);
    sleep(Duration::from_secs(duration)).await;
    println!("Fine task: {}", name);
}

#[tokio::main]
async fn main() {
    // Avvio di due task asincroni
    let task1 = task("Task 1", 2);
    let task2 = task("Task 2", 3);

    // Esecuzione dei task in parallelo
    tokio::join!(task1, task2);

    println!("Tutti i task completati!");
}
```



Attendere la terminazione di più Future

```
use tokio::time::{sleep, Duration};
async fn task(name: &str, duration: u64) -> String {
    println!("Inizio task: {}", name);
    sleep(Duration::from_secs(duration)).await;
    println!("Fine task: {}", name);
    format!("Risultato prodotto da {}", name)
}

#[tokio::main]
async fn main() {
    // Avvio di due task asincroni
    let task1 = task("Task 1", 2);
    let task2 = task("Task 2", 3);

    // Esecuzione dei task in parallelo e lettura dei valori di ritorno
    let (risultato1, risultato2) = tokio::join!(task1, task2);

    println!("Risultato 1: {}", risultato1);
    println!("Risultato 2: {}", risultato2);
}
```



```

use tokio::time::{sleep, Duration};
use std::error::Error;
// Una funzione asincrona che può fallire
async fn task(name: &str, duration: u64, should_fail: bool) -> Result<(), Box<dyn Error>> {
    println!("Inizio task: {}", name);
    sleep(Duration::from_secs(duration)).await;

    if should_fail {
        println!("Task {} fallito!", name);
        Err(format!("Errore nel task: {}", name).into())
    } else {
        println!("Fine task: {}", name);
        Ok(())
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    // Avvio di due task asincroni, uno dei quali fallisce
    let task1 = task("Task 1", 3, false);
    let task2 = task("Task 2", 2, true);
    // Esecuzione dei task in parallelo con gestione degli errori
    match tokio::try_join!(task1, task2) {
        Ok(_) => {
            println!("Tutti i task completati con successo!");
        }
        Err(e) => {
            println!("Uno o più task hanno fallito: {}", e);
        }
    }
    Ok(())
}

```



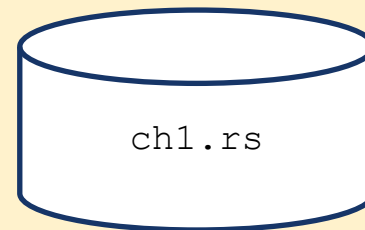
```
use tokio::time::{sleep, Duration};

async fn count() {
    for i in 1..=10 {
        sleep(Duration::from_secs(1)).await;
        println!("Count: {}", i);
    }
}

#[tokio::main]
async fn main() {
    let f1 = count();

    for i in 'a'..='z' {
        print!("{}", i);
    }
    println!();

    f1.await;
}
```

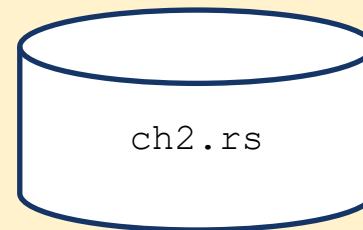


```
use tokio::time::{sleep, Duration};

async fn count() {
    for i in 1..=10 {
        sleep(Duration::from_secs(1)).await;
        println!("Count: {}", i);
    }
}

#[tokio::main]
async fn main() {
    count().await;

    for i in 'a'..='z' {
        print!("{}", i);
    }
    println!();
}
```



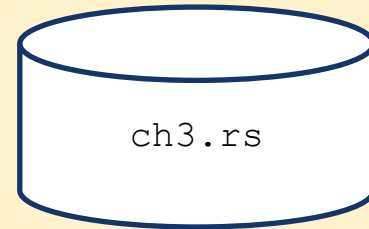
```
use tokio::time::{sleep, Duration};

async fn count() {
    for i in 1..=10 {
        sleep(Duration::from_secs(1)).await;
        println!("Count: {}", i);
    }
}

#[tokio::main]
async fn main() {
    let f1 = tokio::spawn(count());

    for i in 'a'..='z' {
        sleep(Duration::from_millis(500)).await;
        println!("{}", i);
    }

    f1.await.unwrap();
}
```



Sequenza o concorrenza?

- Due strutture diverse, in base alle **dipendenze tra i passi**
- Regola pratica:
 - si introducono **.await** sequenziali solo quando un passo usa un risultato precedente
 - Se due operazioni sono indipendenti, **join!** le sovrappone senza creare task né richiedere **Send**

Selezionare il **primo Future che si completa**

- La macro **tokio::select!(...)** permette di attendere su più rami asincroni, eseguiti nell'ambito dello stesso thread, quello che **termina per primo**
 - I rami non selezionati vengono **cancellati** (viene invocato **Drop** sul loro **Future**)
- Al suo interno è possibile inserire condizioni della forma
 - **<pattern> = <espressione async>(, if <guardia>)? => <gestore>**
 - La guardia opzionale disattiva un ramo quando una preconditione è falsa
 - Un ramo **else** finale viene valutato se nessun ramo corrisponde al pattern indicato

```
use tokio::sync::mpsc;
```

```
#[tokio::main]
```

```
async fn main() {
```

```
    let (tx1, mut rx1) = mpsc::channel::(&str>(10);
```

```
    let (tx2, mut rx2) = mpsc::channel::(&str>(10);
```

```
    tokio::spawn(async move {
```

```
        let _ = tx1.send("Messaggio da canale 1").await;
```

```
    });
```

```
    tokio::spawn(async move {
```

```
        let _ = tx2.send("Messaggio da canale 2").await;
```

```
    });
```

```
    loop {
```

```
        tokio::select! {
```

```
            Some(msg) = rx1.recv() => println!("rx1: {}", msg),
```

```
            Some(msg) = rx2.recv() => println!("rx2: {}", msg),
```

```
            else => {
```

```
                println!("Canali chiusi");
```

```
                break; // entrambi i canali chiusi
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



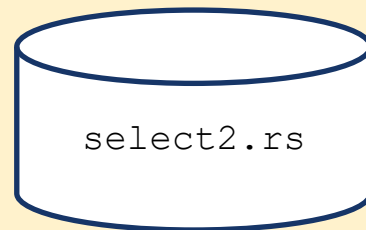
```

use tokio::time::{sleep, Duration};

async fn task(name: &str, seconds: u64) -> String {
    println!("Inizio {name}");
    sleep(Duration::from_secs(seconds)).await;
    format!("{name} completato")
}

#[tokio::main]
async fn main() {
    let usa_task2 = false;
    tokio::select! {
        risultato1 = task("Task 1", 2) => {
            println!("Ha vinto il task 1: {risultato1}");
        }
        risultato2 = task("Task 2", 1), if usa_task2 => {
            println!("Ha vinto il task 2: {risultato2}");
        }
    }
}

```



Gestione del tempo (II)

- La funzione `tokio::time::timeout(d: Duration, f: F).await` attende per un tempo massimo pari alla durata indicata che il Future passato come secondo parametro si completi e restituisce un valore di tipo `Result<T, Elapsed>`
 - Se l'esecuzione si completa in tempo, il risultato è positivo e contiene il valore restituito dal Future, altrimenti riporta un errore di tipo `Elapsed`

```

use tokio::time::{sleep, timeout, Duration};
use std::time::Instant;

async fn task_with_delay() {
    println!("Inizio dell'attività con ritardo");
    // Simula un'attività che richiede 5 secondi per completarsi
    sleep(Duration::from_secs(5)).await;
    println!("Attività completata dopo il ritardo");
}

#[tokio::main]
async fn main() {
    println!("Inizio del programma");

    // Imposta il timeout a 3 secondi
    let timeout_duration = Duration::from_secs(3);

    // Registra il tempo di inizio
    let start_time = Instant::now();

    // Esegue l'attività con un limite di tempo
    let result = timeout(timeout_duration, task_with_delay()).await;

    // Calcola il tempo trascorso
    let elapsed_time = start_time.elapsed();

    match result {
        Ok(_) => println!("Attività completata nei tempi previsti"),
        Err(_) => println!("Timeout! L'attività ha impiegato troppo tempo"),
    }

    println!("Tempo trascorso: {:.2?}", elapsed_time);
}

```

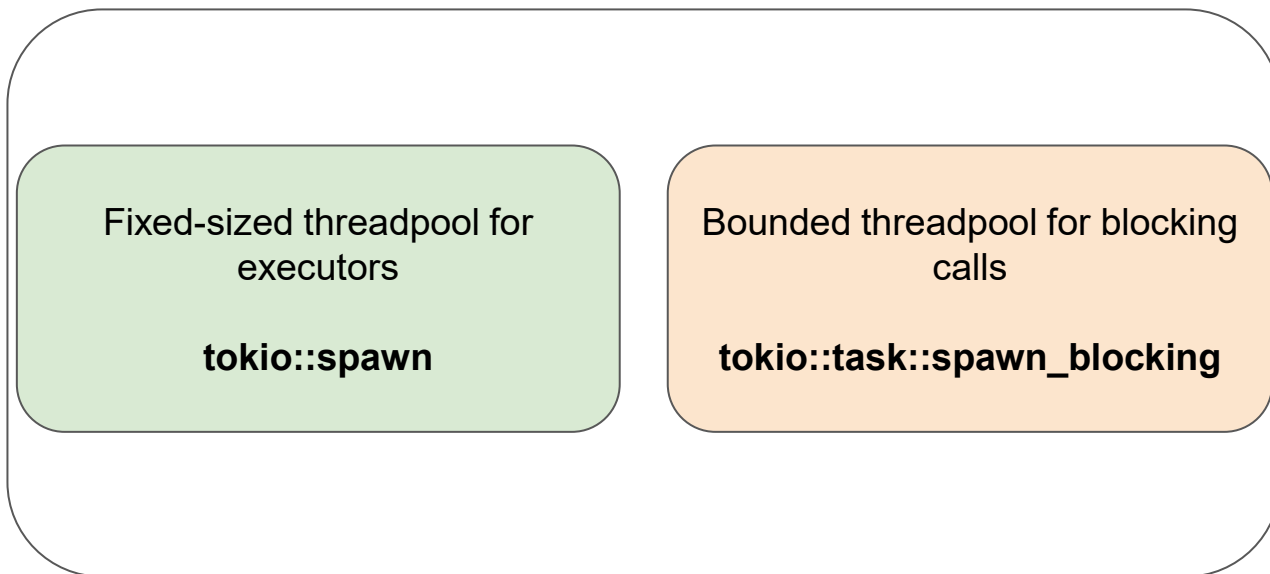


Eseguire compiti computazionalmente intensi

- Se, in una funzione asincrona, occorre eseguire un compito computazionalmente intenso, la cui durata possa superare il centinaio di microsecondi, è bene richiedere che venga eseguito in un thread apposito
 - Così da evitare di introdurre latenza nell'elaborazione degli altri task
 - Si utilizza la funzione `tokio::task::spawn_blocking(f: FnOnce()->R)`
- Occorre evitare di lanciare troppi task di questo tipo
 - Durante la loro esistenza, infatti, tenderebbero a richiedere l'uso di CPU introducendo contesa con i thread deputati a elaborare le code dei messaggi, aumentando la latenza complessiva del sistema
 - Può essere opportuno condizionarne l'esecuzione alla disponibilità di risorse globali o usare esecutori ad hoc, come quelli offerti dalla libreria `Rayon`

Eseguire compiti computazionalmente intensi

Tokio's Runtime

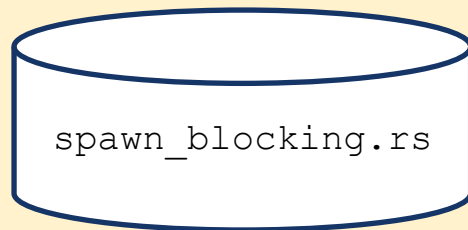


```

use tokio::{task, time::{self, Duration}};
use std::io::{self};
#[tokio::main]
async fn main() -> io::Result<()> {
    let cpu_intensive_handle = task::spawn_blocking(move || {
        println!("[spawn_blocking] Inizio operazione CPU-intensiva (simulata)...");
        let mut sum = 0;
        for i in 0..20_000 { // Un loop lungo per simulare carico CPU
            sum += i;
        }
        println!("[spawn_blocking] Operazione CPU-intensiva terminata.");
        sum // Restituisce il risultato
    });
    let counter_task = tokio::spawn(async {
        for _ in 1..=5 {
            time::sleep(Duration::from_millis(100)).await;
        }
        println!("[async_task] Contatore task terminato.");
    });
    println!("[main] Task in esecuzione");
    let (cpu_result, counter_result) = tokio::join!(cpu_intensive_handle, counter_task);

    match cpu_result {
        Ok(sum_val) => println!("[main] Risultato operazione CPU-intensiva: {}", sum_val),
        Err(e) => eprintln!("[main] Il task CPU-intensivo è fallito: {}", e),
    }
    match counter_result {
        Ok(_) => println!("[main] Il task contatore è stato completato con successo."),
        Err(e) => eprintln!("[main] Il task contatore è fallito: {}", e),
    }
    Ok(())
}

```



Quale costrutto, in quale situazione

Il lavoro è CPU intensive ($> \sim 100\mu s$) ?

→ Thread o Rayon

Passi di I/O che dipendono l'uno dall'altro?

→ `.await` in sequenza

Avviare un'attività indipendente / cancellabile?

→ `tokio::spawn` → `JoinHandle`

Più attività indipendenti, servono tutti i risultati?

→ `tokio::join!` / `try_join!`

Reagire alla prima tra N condizioni?

→ `tokio::select!`

Condividere dati tra task

- Potremmo aver bisogno dei punti di sincronizzazione durante un task e non solo alla fine di un task
- Se due task hanno bisogno di condividere una struttura dati, questa deve essere opportunamente protetta
 - A seguito del meccanismo di *work stealing* adottato dallo schedulatore, è infatti possibile che l'esecuzione avvenga in thread differenti
 - Occorre pertanto adottare le stesse precauzioni e strategie già viste con la programmazione *multithread*
- Tokio mette a disposizione una ricca serie di primitive asincrone nel modulo **`tokio::sync`**
 - Alcune basate sulla condivisione dello stato (**`Mutex`**, **`Semaphore`**, **`Notify`**, **`RwLock`**, **`Barrier`**)
 - Altre basate sulla comunicazione di messaggi (canali **`oneshot`**, **`mpsc`**, **`broadcast`**, **`watch`**)

Condivisione dello stato

- Primitive asincrone basate sulla condivisione dello stato:
 - **Mutex**,
 - **Semaphore**,
 - **Notify**,
 - **RwLock**,
 - **Barrier**.

Arc/Mutex - stato condiviso

```
use std::thread;
use tokio::join;
use tokio::sync::Mutex;
use std::sync::Arc;
#[tokio::main]
async fn main() {
    let data = Arc::new(Mutex::new(0));
    let mut v = vec![];
    for i in 0..4 {
        let data = Arc::clone(&data);
        v.push(tokio::spawn(async move {
            println!("Task {} attivo su {:?}", i, thread::current().id());
            let mut lock = data.lock().await;
            *lock += 1;
            println!("Task {} attivo su {:?}", i, thread::current().id());
        })));
    }
    for h in v { let _ = join!(h); }
    println!("Valore finale è {}", *(data.lock().await));
}
```



```
use std::thread;  
use tokio::sync::Semaphore;  
use tokio::task;  
use std::sync::Arc;  
use std::time::Duration;  
#[tokio::main]
```

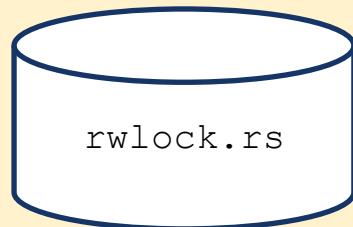
Arc/Semaphores - stato condiviso

```
async fn main() {  
    // Crea un semaforo con 3 permessi iniziali  
    let semaphore = Arc::new(Semaphore::new(3));  
    // Vettore per memorizzare gli handle dei task  
    let mut handles = Vec::new();  
  
    for i in 0..10 {  
        let semaphore_clone = Arc::clone(&semaphore);  
        let handle = task::spawn(async move {  
            // Acquisisce un permesso dal semaforo  
            let permit = semaphore_clone.acquire().await.unwrap();  
            println!("Task {} ha acquisito un permesso {:?}", i, thread::current().id());  
  
            tokio::time::sleep(Duration::from_secs(2)).await; // Simula il lavoro  
  
            println!("Task {} ha rilasciato un permesso {:?}", i, thread::current().id());  
            // Il semaforo viene rilasciato automaticamente quando permit esce dallo scope  
        });  
        handles.push(handle);  
    }  
    for handle in handles { // Attende il completamento di tutti i task  
        handle.await.unwrap();  
    }  
}
```



```
use tokio::sync::RwLock;
use tokio::task;
use std::sync::Arc;
use std::time::Duration;
```

RwLock - stato condiviso



```
#[tokio::main]
async fn main() {
    let lock = Arc::new(RwLock::new(0)); // Crea una RwLock protetta da un Arc
    let lock_clone1 = Arc::clone(&lock); // Clona il lock per il primo lettore
    let reader1 = task::spawn(async move {
        let read_guard = lock_clone1.read().await;
        println!("Lettore 1: il valore è {}", *read_guard);
    });
    let lock_clone2 = Arc::clone(&lock); // Clona il lock per il secondo lettore
    let reader2 = task::spawn(async move {
        let read_guard = lock_clone2.read().await;
        println!("Lettore 2: il valore è {}", *read_guard);
    });
    let lock_clone3 = Arc::clone(&lock); // Clona il lock per lo scrittore
    let writer = task::spawn(async move {
        // Simula del lavoro prima di scrivere
        tokio::time::sleep(Duration::from_secs(1)).await;
        let mut write_guard = lock_clone3.write().await;
        *write_guard += 1;
        println!("Scrittore: il nuovo valore è {}", *write_guard);
    });
    let _ = tokio::join!(reader1, reader2, writer); // Attende il completamento di tutti i task
}
```

```
use tokio::sync::Barrier;
```

```
use tokio::task;
```

```
use std::sync::Arc;
```

```
use tokio::time::{sleep, Duration};
```

```
#[tokio::main]
```

```
async fn main() {
```

```
    let num_tasks = 10; // Numero di task asincroni
```

```
    // Creiamo una barriera condivisa tra i task
```

```
    let barrier = Arc::new(Barrier::new(num_tasks));
```

```
    let mut handles = vec![];
```

```
    for i in 0..num_tasks {
```

```
        let barrier = Arc::clone(&barrier);
```

```
        handles.push(task::spawn(async move {
```

```
            sleep(Duration::from_secs((i+1) as u64)).await;
```

```
            // I task si fermano qui fino a che tutti non hanno raggiunto la barriera
```

```
            println!("Task {} prima della barriera.", i);
```

```
            barrier.wait().await;
```

```
            println!("Task {} dopo la barriera.", i);
```

```
        }));
```

```
    }
```

```
    for handle in handles {
```

```
        handle.await.unwrap();
```

```
    }
```

```
}
```

Barrier - stato condiviso



Notify

- **`tokio::sync::Notify`** è un modulo di sincronizzazione che permette a un task di notificare ad altri task che un certo evento è avvenuto.
- Notify può essere usato per implementare meccanismi di notifica tra task asincroni senza la necessità di un Mutex o altre primitive di sincronizzazione più pesanti.

Notify - stato condiviso



```
use tokio::sync::Notify;
use tokio::task;
use std::sync::Arc;
use std::time::Duration;
#[tokio::main]
async fn main() {
    let notify = Arc::new(Notify::new()); // Crea un oggetto Notify condiviso
    let notify_clone1 = notify.clone(); // Clona il notify per il primo task
    let task1 = task::spawn(async move {
        println!("Task 1: Inizio lavoro");
        // Simula del lavoro asincrono
        tokio::time::sleep(Duration::from_secs(2)).await;
        println!("Task 1: Lavoro completato, notifica task 2");
        // Notifica l'evento
        notify_clone1.notify_one();
    });
    let notify_clone2 = notify.clone(); // Clona il notify per il secondo task
    let task2 = task::spawn(async move {
        println!("Task 2: In attesa di notifica da task 1");
        // Attende la notifica
        notify_clone2.notified().await;
        println!("Task 2: Ricevuta notifica, inizio lavoro");
        // Simula del lavoro asincrono
        tokio::time::sleep(Duration::from_secs(2)).await;
        println!("Task 2: Lavoro completato");
    });
    let _ = tokio::join!(task1, task2); // Attende il completamento dei task
}
```

Nota

- se `notify_one()` viene chiamato **prima** che qualcuno stia aspettando, la notifica non viene persa del tutto. Tokio memorizza **un solo permesso**. La prossima `notified().await` terminerà subito. Però notifiche multiple fatte in anticipo non si accumulano: ne resta solo una.

Uso di Notify

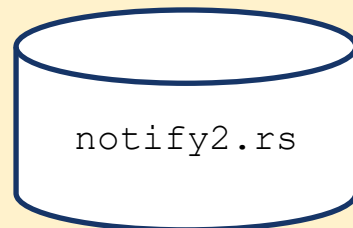
- Notify spesso si usa insieme a una struttura dati condivisa.

```

use std::sync::Arc;
use tokio::sync::{Mutex, Notify};
struct Shared {
    dato: Mutex<Option<String>>,
    notify: Notify,
}
#[tokio::main]
async fn main() {
    let shared = Arc::new(Shared {
        dato: Mutex::new(None),
        notify: Notify::new(),
    });
    // Task consumatore: aspetta che arrivi un dato
    let consumer = shared.clone();
    tokio::spawn(async move {
        consumer.notify.notified().await;
        let dato = consumer.dato.lock().await;
        println!("Ho letto: {}", dato.as_ref().unwrap());
    });
    // Task produttore: scrive il dato e poi sveglia il consumatore
    let producer = shared.clone();
    tokio::spawn(async move {
        let mut dato = producer.dato.lock().await;
        *dato = Some("ciao dal produttore".to_string());
        producer.notify.notify_one();
    });
    tokio::time::sleep(std::time::Duration::from_millis(100)).await;
}

```

Notify - stato condiviso



Comunicazioni tra messaggi

- Primitive asincrone basate sulla comunicazione di messaggi:
 - canali **oneshot**, canale utilizzato per lo scambio di un solo messaggio (consuma poche risorse)
 - canali **mpsc**, canale MPSC di dimensione finita
 - canali **broadcast**, canale che permette di avere più ricevitori dello stesso messaggio
 - canali **watch**, canale che racchiude l'ultimo valore inviato e ad ogni trasmissione permette di informare i receiver che il valore è cambiato.

```
use tokio::sync::oneshot;
use tokio::task;
use std::time::Duration;
```

Canali oneshot - Invio di un solo messaggio

```
#[tokio::main]
async fn main() {
    let num_operations = 5; // Numero di operazioni asincrone
    let mut rxs = Vec::new(); // Vettore per tenere i ricevitori (rx)

    for i in 0..num_operations {
        // Creiamo un nuovo canale oneshot per ogni operazione
        let (tx, rx) = oneshot::channel();

        // Conserviamo il ricevitore
        rxs.push(rx);

        // Spawniamo una nuova task asincrona per l'operazione
        task::spawn(async move {
            // Simuliamo un'operazione che richiede tempo
            tokio::time::sleep(Duration::from_secs(2)).await;
            let result = format!("Result from operation {}", i);

            // Inviemo il risultato attraverso il canale oneshot
            tx.send(result).unwrap();
        });

        // Qui potremmo fare altre operazioni nel task principale
    }

    // Riceviamo i risultati nel task principale
    for rx in rxs.into_iter() {
        let received = rx.await.unwrap(); // quando il valore è stato mandato faccio una await e lo leggo
        println!("Received: {}", received);
    }
}
```



```
use tokio::sync::mpsc;
use tokio::task;
use std::time::Duration;
```

```
#[tokio::main]
async fn main() {
```

```
    // Creiamo un canale mpsc
```

```
    let (tx, mut rx) = mpsc::channel(100);
```

```
    let num_operations = 5; // Numero di operazioni asincrone
```

```
    for i in 0..num_operations {
```

```
        // Cloniamo il trasmettitore per ogni task
```

```
        let tx = tx.clone();
```

```
        // Spawniamo un nuovo task asincrono
```

```
        task::spawn(async move {
```

```
            // Simuliamo un'operazione che richiede tempo
```

```
            tokio::time::sleep(Duration::from_secs(2)).await;
```

```
            let message = format!("Message from task {}", i);
```

```
            // Inviemo il messaggio attraverso il canale mpsc
```

```
            if let Err(e) = tx.send(message).await {
                eprintln!("Failed to send message: {}", e);
            }
        });
    }
```

```
}
```

```
// Chiudiamo il trasmettitore originale per permettere al ricevitore di terminare
```

```
drop(tx);
```

```
// Il task principale riceve i messaggi
```

```
while let Some(received) = rx.recv().await {
```

```
    println!("Received: {}", received);
}
```

```
}
```

```
println!("All tasks completed and channel closed.");
```

```
}
```

Canali mpsc - Multiple Producer Single Consumer



```
use tokio::sync::mpsc;
use tokio::time::{self, Duration};
#[tokio::main]
async fn main() {
    let (tx1, mut rx1) = mpsc::channel(32);
    let (tx2, mut rx2) = mpsc::channel(32);

    // Simula la trasmissione di messaggi dopo un certo tempo
    tokio::spawn(async move {
        time::sleep(Duration::from_secs(2)).await;
        tx1.send("da tx1").await.unwrap();
    });
    tokio::spawn(async move {
        time::sleep(Duration::from_secs(2)).await;
        tx2.send("da tx2").await.unwrap();
    });
    tokio::select! {
        Some(msg) = rx2.recv() => {
            println!("Ricevuto: {}", msg);
        }
        Some(msg) = rx1.recv() => {
            println!("Ricevuto: {}", msg);
        }
        _ = time::sleep(Duration::from_secs(1)) => {
            println!("Timeout raggiunto");
        }
    }
}
```




```
use tokio::sync::mpsc;
use tokio::task;
use tokio::time::{self, Duration};
```

```
#[tokio::main]
async fn main() {
```

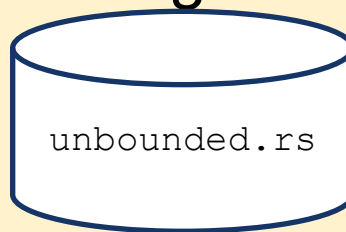
Canali mpsc - Multiple Producer Single Consumer

```
    // Create an unbounded channel
    let (tx, mut rx) = mpsc::unbounded_channel();

    // Spawn a task to send messages
    let tx_task = task::spawn(async move {
        for i in 0..10 {
            if tx.send(i).is_err() {
                println!("Receiver dropped");
                return;
            }
            println!("Sent: {}", i);
            time::sleep(Duration::from_millis(500)).await;
        }
    });

    // Spawn a task to receive messages
    let rx_task = task::spawn(async move {
        while let Some(msg) = rx.recv().await {
            time::sleep(Duration::from_millis(2000)).await;
            println!("Received: {}", msg);
        }
    });

    // Wait for both tasks to complete
    let _ = tokio::join!(tx_task, rx_task);
```



Canali broadcast - comunicazione multi-molti

```
#[tokio::main]
async fn main() {
    let (tx, mut rx1) = broadcast::channel(16);
    let mut rx2 = tx.subscribe(); // serve per far sapere al trasmettitore
                                   // che sto clonando il ricevitore
                                   // il dato è tolto dal canale solo quando
                                   // l'hanno letto tutti i ricevitori

    tokio::spawn(async move {
        assert_eq!(rx1.recv().await.unwrap(), 10);
        assert_eq!(rx1.recv().await.unwrap(), 20);
    });
    tokio::spawn(async move {
        assert_eq!(rx2.recv().await.unwrap(), 10);
        assert_eq!(rx2.recv().await.unwrap(), 20);
    });
    tx.send(10).unwrap();
    tx.send(20).unwrap();
}
```

```

use tokio::sync::broadcast;
use tokio::task;
use tokio::time::{sleep, Duration};
#[tokio::main]
async fn main() {
    let (tx, _rx) = broadcast::channel(16);

    // Spawn multiple receiver tasks
    for i in 0..3 {
        let mut rx = tx.subscribe();
        task::spawn(async move {
            while let Ok(msg) = rx.recv().await {
                println!("Receiver {} got: {}", i, msg);
            }
        });
    }

    // Sender task
    let tx_task = task::spawn(async move {
        for i in 0..5 {
            if tx.send(i).is_err() {
                println!("No active receivers");
                return;
            }
            println!("Sent: {}", i);
            sleep(Duration::from_secs(1)).await;
        }
    });

    let _ = tx_task.await; // Wait for the sender task to complete
}

```



broadcast1.rs

Canali watch - pattern Observer

- Un canale multi-producer e multi-consumer che conserva solo l'ultimo valore inviato
- **Il channel è creato con un valore iniziale**
- Per accedere al valore corrente memorizzato nel canale e contrassegnarlo come visto da un determinato ricevitore, si utilizza `Receiver::borrow_and_update()`.
- Per accedere al valore corrente senza contrassegnarlo come visto, si utilizza `Receiver::borrow()`.

Canali watch - pattern Observer

```
use tokio::sync::watch;
use tokio::time::{Duration};
#[tokio::main]
async fn main() {
    let (tx, rx) = watch::channel("default");

    for _ in 0..2 {
        let mut rx = rx.clone();
        tokio::spawn(async move {
            while rx.changed().await.is_ok() {
                println!("received: {:?}", *rx.borrow());
            }
        });
    }
    let d = Duration::from_secs(1);
    tx.send("value 1").unwrap();
    tokio::time::sleep(d).await;
    tx.send("value 2").unwrap();
    tokio::time::sleep(d).await;
}
```



```
use tokio::sync::watch;
use tokio::time::{Duration, sleep};
#[tokio::main]
async fn main() {
    let (tx, mut rx) = watch::channel("Prendi questo ed esci quando qualcuno lo cambia");

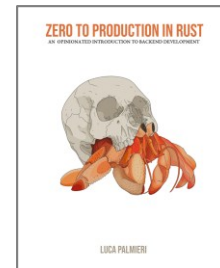
    tokio::spawn(async move {
        // Use the equivalent of a "do-while" loop so the initial value is
        // processed before awaiting the `changed()` future.
        loop {
            println!("{}", *rx.borrow_and_update());
            if rx.changed().await.is_err() {
                break;
            }
        }
    });

    sleep(Duration::from_millis(100)).await;
    tx.send("EOF").unwrap();
}
```



Link

- Asynchronous Programming in Rust
 - <https://rust-lang.github.io/async-book/>
 - Trattazione dettagliata dei principi relativi alla programmazione asincrona in Rust
- Fearless Concurrency with Rust
 - <https://medium.com/pragmatic-programmers/fearless-concurrency-with-rust-part-3-asynchronous-concurrency-e23bad856087>
 - Trattazione esemplificativa dell'elaborazione asincrona applicata ad un server web
- What's a “Thread Boundary” in Rust's Async-Await ?
 - <https://cotigao.medium.com/whats-a-thread-boundary-in-rust-s-async-await-f783cff55c99>
 - Approfondimento sul meccanismo di *work stealing* usato in Tokio
- Zero to Production in Rust
 - L. Palmieri, 2022, ISBN 979-8847211437
 - <https://www.zero2prod.com/>
 - Introduzione pratica e dettagliata allo sviluppo di componenti backend in Rust



Glossario

- Computazione CPU-bound: processo che utilizza intensivamente la CPU
- Computazione I/O-bound: processo che passa la maggior parte del tempo ad aspettare i dati che arrivano da I/O
- Funzione di callback: funzione eseguita solo dopo che l'operazione asincrona (bloccante) è terminata. Viene passata come parametro alla funzione asincrona
- Future: computazione asincrona (non bloccante) che potrebbe non essere ancora terminata e che alla fine produrrà un valore (o un errore)
- Task: un'unità di lavoro concorrente che può essere eseguita in modo non bloccante.